

Answers and Solutions

Exam #1 ~ Multiple Choice

1. B	11. D	21. C	31. D	41. C
2. C	12. A	22. A	32. B	42. A
3. B	13. B	23. B	33. B	
4. C	14. C	24. C	34. D	
5. C	15. D	25. A	35. D	
6. C	16. B	26. B	36. A	
7. D	17. D	27. C	37. A	
8. A	18. A	28. B	38. D	
9. B	19. D	29. D	39. D	
10. A	20. B	30. C	40. C	

Notes:

1. “&& z” in Choice B gives the answer away.
2. In Choice A, `(double) (p * q / 2)` evaluates to 7.0; in Choice B, `(double) (q / 2)` evaluates to 1.0; Choice C is correct.
3. The smallest possible value of `x` is 0, when `Math.random()` returns a number close to 0 (any number $< 1/16$). The largest possible value of `x` is 2, when `Math.random()` returns a number close enough to 1 (any number $> 9/16$).
4. `list.remove(i)` shifts the subsequent elements to the left by one, so after successive calls to `list.remove(i)` only every other element of `list` is removed.
5. `product = (3%2) * (7%4) * (5%2) = 1 * 3 * 1`
6. `\\` represents one backslash, and `\n` represents the newline character. So `str` actually consists of 5 characters: `\`, `*`, *newline character*, `*`, `\`.
7. `count` is not incremented.
8. The expression in Choice A correctly assigns to the index a random value from 0 to `sample.length - 1`, inclusive.
9. All the lines are okay due to autoboxing and unboxing. `n + x` is converted to a double, and `println` displays the double value with a decimal point.
10. Apply a De Morgan’s Law.
11. `4/3` yields 1

12. $12\%7$ gives 5; `(double) (12/5)` results in 2.0; `n` remains unchanged, because it is passed to `goFigure` by value.
13. The for-each loop in Choice A does not work, because `word` is a copy of the string in `words`. The copy is modified but the value in the list remains unchanged. Choice C is trickier: at first it seems it might work, but actually it does not. Suppose `words` has only two elements, `["A", "B"]`. After the first iteration, `words` becomes `["B", "A*"]` and `k` becomes 1; after the second iteration, the code throws an `IndexOutOfBoundsException`. To make it work, replace `words.add(k+1)` with `words.add(k)`.
14. For each element `p[i]`, `mysteryCount` adds to `count` the length of the cycle `p[p[p...[i]...]] = i`. This is 1 for the first element and 3 for each of the other three elements.
15. In Choice A, `"Tick-Tock"` is equal to `"Tick" + "-" + "Tock"`. In Choice B, `"Tick"` is less than `"Tock"`. In Choice C, `"Tick-Tock".indexOf("t")` is -1.
16. When `words` is created, its size is 0. Trying to access the first element in `words` causes an `IndexOutOfBoundsException`.
17. The code segments in Choices A, B, and C fail for the same reason: in all three of them `"!"` is appended to a local variable, not to an element of `letters`.
18. $0 - 1 + 2 - 3 + 4 - \dots + 10 = 0 + (-1 + 2) + (-3 + 4) + \dots + (-9 + 10) = 5$
19. `BuddyList myFriends` might not have a `get` method, and `buddies` in a `BuddyList` is not an array. Even if it were, `buddies` is `private` in `BuddyList` and not directly accessible in client classes. This eliminates Choices A and B. The same privacy restriction eliminates Choice C.
20. At the end of an iteration, `i` is 3, then 4, then 5; `k` changes from 0 to 3 to 7 to 12.
21. Three times for $p = 2$; two times for $p = 3$; one time for $p = 4$; two times for $p = 5$.
 $3 + 2 + 1 + 2 = 8$
22. `n` remains unchanged, because it is passed to `change` by value. The `change` method does change the values in `samples`, because the method gets a copy of the reference to `samples`.
23. `filter` makes a new string from `str`, with all the occurrences of `pattern` in `str` removed one by one.
24. The code in Choice C does not work, because in the second for-each loop, `x` acts like a local variable; you cannot replace the value of an array element in a for-each loop.
25. After the first pass through the outer loop: `counts` holds 0, 2, 3, 4, 5, 1; 1 swap. After the second pass through the outer loop: `counts` holds 0, 1, 3, 4, 5, 2; 1 swap. After the third pass through the outer loop: `counts` holds 0, 1, 2, 4, 5, 3; 1 swap. And so on.

-
26. Encapsulation, which is also called information hiding.
 27. If you add or remove elements while traversing an `ArrayList` with a for-each loop, the code will throw a `ConcurrentModificationException`.
 28. $4 + 9 + 2 + 5 + 7 + 6 = 33$
 29. It is often said that a for-each loop cannot change the values in a list. This is true only when the list holds values of a primitive data type or immutable objects. A mutable object in the list can be changed by calling one of its mutator methods. Here the list first holds four bank accounts with balances 0, 5, 10, and 15. Then 10 is added to each balance.
 30. When one of the operands in the `+` operator is a string, the other is converted to a string, and the two strings are concatenated. The `+` operators are applied from left to right.
 31. If `targetValue` is not in `a`, the `while` loop eventually causes an `ArrayIndexOutOfBoundsException`.
 32. When the k -th element is removed from `list` in the second `for` loop, the subsequent elements are shifted to the left, and their indices are decremented by one. As a result, this loop removes every other element — values 1, 3, and 5, at the original indices 0, 2, and 4. These values are added at the end of the list.
 33. A `Monster` object keeps track of the biggest (lexicographically) string passed to its `feed` method. In this case it happens to be "Two". `x` is initialized to `null` by default, then updated as necessary to subsequent strings. `NullPointerException` does not occur, due to short-circuit evaluation.
 34. In the first iteration through the outer `for` loop, `count` is set to 1, then multiplied by 2 three times. It becomes 8. In the second iteration through the outer `for` loop, 1 is added to `count`, which is then multiplied by 2 three times. `count` becomes $9 * 8 = 72$.
 35. `Scanner` treats a blank line as any other line; `hasNext` returns `true`, and `nextLine` returns an empty string. `next` skips all "white space" (spaces, tabs, newline characters).
 36. Binary Search will look at `arr[63]`, `arr[31]`, `arr[47]`, and `arr[39]`.
 37. `arr[0]` is never changed, which leaves us with Choices A and B. On the last iteration, `arr[7]` is assigned the value of `arr[5]`, which is 6.
 38. The for-each loops in Choices A and B are wrong because `m` is not a one-dimensional array. Choice C is wrong because the outer loop has to iterate over the first index (the Java convention is that the first index is row, the second is column).
 39. In Choice A, `nextLine` is called twice for each line. In Choice B, the line does not have to be equal to `target`, only contain it. In Choice C, lines will be squashed together on a single line in the output, because they are printed with `print`, not `println`. Unlike input handling in some other programming languages, Java's `nextLine` consumes the newline character at the end of the line, but it does not include it in the returned string.

40. The nested `for` loop correctly traverses the entire array and fills all its elements in a diagonal pattern, starting with 1 in the upper-left corner.
41. This is basically a Binary Search algorithm. We need 1 iteration for one element, 2 iterations for three elements, 3 iterations for seven elements, 4 iterations for 15 elements, and 5 iterations for 31 elements. There is no “best” or “worst” case.
42. The code in Choices B, C, and D creates a new object of the type `Test` and works with it. All three will compile fine and display 2026. (In Choice C, `first year` is set to 26, then it is concatenated with 20.) The code in Choice A will not compile, because `this`, the reference to an object whose constructor or method is running, is undefined in a static method, such as `main`.

Answers and Solutions

Exam #1 ~ Free Response

1. (a)

```
public int assignedPercentage()
{
    int total = 0, vacant = 0;

    for (int room = 1; room <= numRooms; room++)
    {
        int beds = totalBeds(room); 1
        if (beds > 1)
        {
            total += beds;
            vacant += vacantBeds(room); 1
        }
    }
    return (int)(100.0 * (total - vacant)/total + 0.5);
}
```

Notes:

1. Call the provided methods.

(b)

```
public Dorm(String map)
{
    dormMap = map;
    numRooms = 0;

    int i = 0;
    map = map.substring(i+1);

    while (i != - 1)
    {
        i = map.indexOf("/"); 1
        if (i != -1)
        {
            numRooms++;
            map = map.substring(i+1);
        }
    } 2
}
```

Notes:

1. An alternative solution can use `map.indexOf("/", i)` and avoid updating `map`.
2. A one-liner solution would use `map.split("/")`.

2.

```
public class ReviewedProduct
{
    private String name;
    private int totalReviews = 0;
    private int oneStarReviews = 0;
    private int totalStars = 0;

    public ReviewedProduct(String nm)
    {
        name = nm;
    }

    public String addReview(int numStars)
    {
        totalReviews++;
        totalStars += numStars;

        if (numStars == 1)
            oneStarReviews++;

        if (totalReviews == 1)
        {
            return name + " has been reviewed only once";
        }
        else if (totalReviews >= 3 &&
                 2*oneStarReviews > totalReviews)
        {
            return name + " has been reviewed " +
                totalReviews + " times; too many one-star reviews";
        }
        else
        {
            double rating = 0.5 * (int) (2.0 * totalStars /
                                         numStars + 0.5);1
            return name + " has been reviewed " + totalReviews +
                " times; average rating " + rating + " stars";
        }
    }
}
```

Notes:

1. This is how you can round the ratio to the nearest half-star.

3.

```
public boolean openTwoCards(int k1, int k2)
{
    for (int i = 0; i < cards.size(); i++)
    {
        int count = 0;

        for (int j = 0; j < cards.size(); j++)
        {
            if (cards.get(i).equals(cards.get(j))) 1
                count++;
        }
        if (count != 2) 2
            return false;
    }

    if (cards.get(k1).equals(cards.get(k2)))
    {
        cards.remove(k2);
        cards.remove(k1); 3
        return true;
    }
    else
    {
        return false;
    }
}
```

Notes:

1. Use equals, not ==.
2. count should be 2, not 1, because each card is equal to itself.
3. It is important to remove the card with the greater index first.

4.

```
public void moveAllForward()
{
    int lanes = hwy.length;

    for (int row = 0; row < lanes; row++)
    {
        for (int col = hwy[0].length - 1; col > 0; col--)
        {
            hwy[row][col] = hwy[row][col-1];
        }

        if (Math.random() <= probability)
            hwy[row][0] = 1;
    }
}
```


Answers and Solutions

Exam #2 ~ Multiple Choice

1. B	11. C	21. A	31. B	41. D
2. D	12. B	22. C	32. D	42. C
3. B	13. A	23. B	33. D	
4. A	14. D	24. A	34. C	
5. D	15. A	25. C	35. D	
6. A	16. B	26. D	36. C	
7. D	17. B	27. A	37. C	
8. C	18. C	28. A	38. C	
9. A	19. C	29. D	39. A	
10. A	20. B	30. B	40. C	

Notes:

- Both `c` and `d` become zeros, because the integer division is performed first.
- Convert the meaning of each expression into words or, more formally, try `x = true`, `y = false`, then `x = true`, `y = true`. (No need to try `x = false`, `y = true` because each of the Choices A, B, and C represents a “symmetric” expression with respect to `x` and `y`.)
- Cannot convert a `String` into an `Integer` this way — not allowed in Java. You need to use

```
Integer x = Integer.parseInt("123");
```
- `\\` in `"yes\\no"` represents one backslash character, and `\n` in `"\no"` represents the newline character. The `"yes\\no"` string does not include a newline character.
- Once the first positive integer is found and swapped with the first element of `arr`, no further changes to `arr` are made.
- The method finds the greatest common divisor of `a` and `b`, which in this case is 3. Initially, `d` is set to the smaller of `a` and `b`, in this case 12. Then the values of `d` are tested until 3 is found. You might be tempted to conclude that two `%` operations are performed for each number: $2*10 = 20$. But due to short-circuit evaluation, two `%` operations will be performed only for the values of `d` that are divisors of 12: 12, 6, 4, and 3. Only one `%` operation will be performed for the remaining values: 11, 10, 9, 8, 7, 5. $2*4 + 6*1 = 14$
- The code compiles fine due to autoboxing on Line 1, unboxing on Line 2, and autoboxing on Line 3. The result of calling `Math.pow` is rounded to the nearest integer.

8. The boolean variable `flag` “accumulates” the values of the Boolean expressions `v[i] == v[i+1]`; that is, it remains `true` once one of these expressions is `true`.
9. `list.remove(i)` shifts all the subsequent elements to the left. Therefore, it is a mistake to increment `i` when an element is removed: the subsequent element won’t be examined. This common bug, related to `ArrayLists`, manifests itself when two consecutive elements in the list satisfy the condition for removal.
10. Choice C refers to a situation in which either an array holds values of a primitive data type, or an array or `ArrayList` holds immutable objects. In these situations you cannot change values in the array or list within a for-each loop. However, if objects in the list are mutable, as the `Turtle` objects are here, you can change an object by calling its mutator method. The proposition in Choice D is simply not true for an `ArrayList` (and the opposite is true for a `LinkedList`).
11. $(m + n) / 2$ evaluates to 4.
12. `n != 10` is the correct choice for *< statement 2 >* (to eliminate the rightmost digit). This leaves only Choices A and B. If $n = 0$, the return should be 1 only when $d = 0$, so Choice A is wrong.
13. By one of the De Morgan’s Laws, this is equivalent to `(a && !b) || (!a && b)`.
14. `band.substring(1, 4)` is “NAM”; `band.substring(5, 8)` is “NAG”.
15. The last iteration starts with `num = 1`.
16. `r` does not have to be a double (`x/y` results in an `int` anyway); `r` can be either a static variable or an instance variable in `ClassX`.
17. `numbers` holds two elements: two copies of an `Integer` with value 1. `names` holds `["Cathy", "Ben", "Any"]`. Then the element at index 1 is deleted twice from `names`.
18. Each of the two addends can have the value 0 or 1, independently of the other. So, their sum can have the value 0, 1, or 2 (actually, 1 is twice as likely to occur as 0 or 2).
19. `nums[2]` is not assigned a value, so it remains `null`. A call to `null.toString()` raises a `NullPointerException`.
20. The addition of integers is performed first.
21. First, `a` and `b` are passed to `swap` by value, so `swap` cannot change their values. This rules out Choices C and D. Inside `swap`, `a` and `b` act as local variables. The assignment `b = a` makes them equal, so `swap` returns 0.
22. 4 times for each pair plus 2 times for each quadruplet plus 1 time at the top level.
23. The code adds 2, 4, 8, 16, and 32 to the elements of `v`, respectively. `v[4]`, the last element, becomes 33.

24. Java does not catch arithmetic overflow errors — it just prints an incorrect result. Therefore, the statement compiles with no problems, and Choices C and D do not apply. `Integer.MAX_VALUE` is the largest possible value of the `int` type that can be printed, and the value of $2 * \text{Integer.MAX_VALUE} + 2$, which equals 2^{32} , exceeds the `int` range, so Choice B is wrong. This leaves Choice A as the answer. (In Java, an `int` value is always held in four bytes, which is 32 bits. The result here is 0 because the high-order bit in the binary representation of 2^{32} — which is 1 followed by 32 zeros — is lost and only the 32 zeros remain.)
25. Since `mystery(4)` calls `mystery(3)`, and `mystery(3)` calls `mystery(2)`, and `mystery(2)` calls `mystery(1)`, it is easier to compute the result of calling `mystery(4)` by starting at the bottom. `mystery(1)` returns 3; `mystery(2)` returns $3 * 3 + 1 = 10$; `mystery(3)` returns $3 * 10 + 1 = 31$; `mystery(4)` returns $3 * 31 + 1 = 94$.
26. `s.substring(1, 3)` is "LA"; "S" is not in it.
27. A constructor does not return a value and has no return type. This eliminates Choices B, C, and D. The answer is A. (This no-argument constructor does nothing, which is allowed.)
28. `examine` returns `true` even when all the letters in the array are different, because in the inner `for` loop each letter is also compared to itself.
29. `a` and `b` refer to the same array. After the assignments, its values are `a[0] = b[0] = 2`; `a[1] = b[1] = 1`.
30. In Choice B, you cannot access an instance variable `score` from a static method. (You can, of course, access static variables in constructors and instance methods.)
31. Let's trace the values of the variables `a` and `b`:
- | | <code>a</code> | <code>b</code> |
|----------------------------------|----------------|----------------|
| Before the <code>for</code> loop | 1 | 2 |
| After the first iteration | 2 | 3 |
| After the second iteration | 3 | 2 |
- (Interestingly, this sequence of `a`, `b` pairs is periodic, with the period 5.)
32. A `for-each` loop cannot change the values in an `ArrayList<String>` because `String` objects are immutable.
33. The last thing the call to `splat("***")` does is display `**` on a separate line, which eliminates Choice B. `splat("***")` calls `splat("*****")`, which in turn calls `splat("*****")`. As `splat("*****")` finishes its work first, the first line printed is `*****`. Then `splat("*****")` finishes and prints `****`. Last, `**` is printed, as discussed above.
34. The code in Choice A fails because `nextInt` only works with integers separated by white space (spaces, tabs, newline characters). In the code in Choice B, `next` will read the entire line. `next` reads tokens separated by white space. The `while` loop in Choice D should have `hasNext`, not `nextLine`.

35. After the first iteration through the outer for-each loop, with $f = 2$, `nums` will hold `[1, 2]`. After the second iteration, with $f = 3$, `nums` will hold `[1, 2, 3, 6]`. After the third iteration, with $f = 5$, `nums` will hold `[1, 2, 3, 6, 5, 10, 15, 30]`.
36. This is an implementation of a standard algorithm for reversing an array.
37. We have to add to `sum` only odd numbers, starting with 1 and ending with $2n-1$; that is, $2k-1$ for k from 1 to n .
38. This shuffling algorithm is similar to Selection Sort. In each iteration through the `while` loop, a random card among the first n is selected and swapped with the n -th card.
39. This is a standard implementation of Insertion Sort.
40. The base case, $n \leq 2$, is implicit in this method: the `for` loop will never be entered. So, `fun(2)` returns 2. `fun(3)` returns $2 \cdot 2 = 4$. `fun(4)` returns $2 \cdot 2 \cdot 4 = 16$. `fun(5)` returns $2 \cdot 2 \cdot 4 \cdot 16 = 256$.
41. The value of an element in a given column is set to the sum of the values of its two or three neighbors in the previous column. It is sufficient to look at the first two iterations.
42. Choices A and B are wrong. Privacy applies to the source code of a class, and any object of the class has access to private instance variables and methods of any other object of the class.

(It might seem fanciful to think of a subclass not using the instance variables of its superclass. However, the “Decorator” design pattern suggests just that. This pattern would use a “wrapper” class, say `DecoratedAthlete`, that both extends `Athlete` and has an `Athlete` object as an instance variable. A method of `DecoratedAthlete` calls the corresponding method of the embedded `Athlete` object, while the instance variables of `DecoratedAthlete` itself — other than the embedded `Athlete` — are unused.)

Answers and Solutions

Exam #2 ~ Free Response

1. (a)

```
public void makeAwards()
{
    for(int year = 1950; year <= currentYear(); year++)
    {
        String award = " ";

        while (award != null)
        {
            award = getAward(year);

            if (award != null)
            {
                if (allAwards.length() > 0)
                    allAwards += ", ";

                allAwards += year + " " + award;
            }
        }
    }
}
```

(b)

```
public String reachedEgot()
{
    String awards = allAwards + ", ";1, 2
    String egot = "";
    int year = 0;
    int i = 0;

    while (i != -1 && egot.length() < 4)
    {
        i = awards.indexOf(", ");
        if (i != -1)
        {
            year = Integer.parseInt(awards.substring(0, 4));
            String letter = awards.substring(5, 6);

            if (egot.indexOf(letter) == -1)
                egot += letter;

            awards = awards.substring(i + 2);3
        }
    }

    if (egot.length() == 4)
        return "Reached EGOT status in " + year;
    else
        return currentYear() + " Not EGOT yet";
}
```

Notes:

1. allAwards is declared final. In this solution we have to make a copy.
2. ", " added to allow uniform handling of the last word.
3. Rather than updating the awards string, we could use allAwards.indexOf(", ", i).

2.

```
public class Shopping
{
    private final int MAX_DISTANCE;
    private String bestStore;
    private double bestPrice;

    public Shopping(int distance)
    {
        MAX_DISTANCE = distance;
        bestStore = null; // optional (default)
    }

    public String addShoppingOption(String name,
                                    double price, int distance)
    {
        if (distance <= MAX_DISTANCE &&
            (bestStore == null || price < bestPrice))
        {
            bestStore = name;
            bestPrice = price;
        }

        return bestStore + " price " + bestPrice;
    }
}
```

3.

```
public static void removeOutliers(ArrayList<Movie> movies)
{
    ArrayList<Double> coefficients =
        getFitCoefficients(movies);

    double sum = 0.0;

    for (Double c : coefficients)
        sum += c; 1

    double threshold = (sum / coefficients.size()) / 2;

    for (int k = movies.size() - 1; k >= 0; k--) 2
    {
        if (coefficients.get(k) < threshold)
        {
            movies.remove(k);
        }
    }
}
```

Notes:

1. Unboxing.
2. It is important to scan in reverse order, because when a movie is removed, the subsequent movies shift toward the beginning of the list.

4.

```
public boolean isPacked()
{
    for (int row = 1; row < tiles.length; row++)
    {
        for (int col = 0; col < tiles[0].length; col++)
        {
            if (tiles[row][col] != null &&
                tiles[row-1][col] == null)
                return false;
        }
    }

    for (int col = 1; col < tiles[0].length; col++)
    {
        for (int row = 0; row < tiles.length; row++)
        {
            if (tiles[row][col] != null &&
                tiles[row][col-1] == null)
                return false;
        }
    }

    return true;
} 1
```

Notes:

1. An alternative solution:

```
public boolean isPacked()
{
    for (int row = 0; row < tiles.length; row++)
    {
        for (int col = 0; col < tiles[0].length; col++)
        {
            if (tiles[row][col] != null)
            {
                if (row > 0 && tiles[row-1][col] == null)
                    return false;

                if (col > 0 && tiles[row][col-1] == null)
                    return false;
            }
        }
    }

    return true;
}
```


Answers and Solutions

Exam #3 ~ Multiple Choice

1. C	11. D	21. A	31. A	41. B
2. C	12. D	22. C	32. B	42. C
3. D	13. B	23. C	33. B	
4. A	14. D	24. D	34. C	
5. B	15. A	25. B	35. A	
6. D	16. B	26. B	36. C	
7. A	17. D	27. B	37. D	
8. A	18. B	28. D	38. D	
9. C	19. D	29. D	39. C	
10. A	20. A	30. C	40. B	

Notes:

1. Suppose `somethingIsFalse` returns `true`. Then the `else` clause is executed in the code segment and it returns `true`, and so does the statement in Choice C.
2. `abc1` is first set to `"AAABBBCC"`, then to `"AABBBCC"`; it is a substring of `abc`, starting at index 1.
3. $1 < x/y < 2$ and $4 < x*y < 5$; truncating to integers gives $1 + 4 = 5$.
4. Draw a truth table for each of the two expressions to show that none of their respective rows are the same.
5. The statements compile fine. In the first line, 2026 is converted to an `Integer` object due to autoboxing. In the second line, in the sum expression, `year` is converted back to an `int` with the value 2026, due to unboxing. `Integer.parseInt("1")` simply gives 1.
6. `b` is set to 0; `a` remains 3; `a/b` in the condition is not evaluated, due to short-circuit evaluation.
7. The expression combines both De Morgan's Laws with the `&&` operator.
8. `\\` represents one backslash; `\` represents a quotation mark; `substring(1)` gets rid of the leading backslash.

9. Trace the values of the variables:

n	d	s
1	1	1
2	3	4
3	5	9
4	7	16
5	9	25

The loop runs five times, for $n = 1, 2, 3, 4, 5$.

10. `arr` is passed to `reduce` as a reference to the original array; `len` is passed by value. Notice that `reduce` decrements only the first `len` elements of `arr`.
11. In Choice B, `START_POS` cannot be changed because it is declared `final`.
12. The two consecutive calls to `randomPoints(3)` may return different numbers in the range from 1 to 3, inclusive.
13. Array indices start from 0; need for `arr[i]` to be both positive and odd.
14. Simplify using a De Morgan's Law and notice the short-circuit evaluation.
15. If you try to add an element to an `ArrayList` or remove an element from an `ArrayList` inside a for-each loop, the program throws a `ConcurrentModificationException`.
16. The ratio is a negative number between -2 and -1 ; it is truncated toward 0.
17. `list[0]`, not `list[2]`, holds a reference to the first `APScholar`, so Choice B is off base. Choice C cannot be right because the `getExamResult` method, not the `getScore` method, takes an integer parameter. Choice A is more plausible, but it doesn't work, because `exams` is private in `APScholar`.
18. When the `+` operator is applied to a string and an integer, the integer is converted to a string, so there is no syntax error on Line 1. The `==` operator, when applied to two objects, compares the addresses of the objects, which in this case are different. `c.equals(d)` compares the values of `c` and `d`, which are the same.
19. Starting with `[0, 1]`, each iteration through the outer loop doubles the size of the list `numbers`. After three iterations the size becomes 16. This eliminates Choices A and B. Also, in each iteration through the outer loop, the value of the last element of `numbers` is incremented by 1. After three iterations, the last element becomes 4.
20. `b = fun2(a, b)` sets `b` to 4; `a` remains 3 (because `a` and `b` are passed to `fun2` by value); then `a = fun2(b, a)` sets `a` to -1 ; `b` remains 4.
21. With 160,000 numbers, it will take 18 ms to sort each half. Then, merging the two sorted halves with 80,000 numbers in each of them takes $40 - 2 \cdot 18 = 4$ ms. For 320,000 elements, it will take $2 \cdot 40$ ms to sort each half and $2 \cdot 4$ ms to merge the sorted halves with 160,000 numbers in each, for a total of $2 \cdot 40 + 8 = 88$ ms.
22. `Math.random` takes no parameters, so only Choices C and D remain. `Math.random` returns a random double in the range $0 \leq x < 1$. In Choice D, "A" will never be chosen.

-
23. `lol` is called in succession with parameters 4, 3, 2, 1, and 0.
24. A blank line is treated the same way as any other line: `hasNext` returns `true` and `nextLine` returns an empty string.
25. For the expression to evaluate to 0, the sum of the values returned by the two `Math.random()` calls must be less than 0.5. This is an unlikely event (to be precise, its probability is 1/8). Similarly, the probability that the sum of the values returned by the two `Math.random()` calls is greater than or equal to 1.5 is also low (also 1/8). The probability of getting 1 is much greater (it is $1 - 2/8 = 3/4$).
26. The `while` loop finds the correct spot to insert `current`; the inner `for` loop shifts the elements in `a` to the right to make room.
27. The last element remains unchanged, which eliminates Choice A.
28. `s` keeps growing until it overflows the memory: "ABC", "ABCABC", "ABCABCBCABC", and so on.
29. `list1` and `list2` refer to the same list. It first becomes `[1]`, then `[2, 1]`.
30. The statement `int n = str.length()` will raise a `NullPointerException`.
31. The data type of `Integer.MAX_VALUE` is `int`, so `Integer.MAX_VALUE / 10` is truncated to an integer and gives 214748364. `Integer.MAX_VALUE / 10 + 1` gives 214748365.
32. The optimal search algorithm here is Binary Search. Three comparisons are needed if we don't know whether or not "Desiree" is in the `names` array.
33. `yx` and `xy` refer to the same array, so `yx[0]` and `xy[0]` are both equal to "Y" after the assignment `yx[0] = xy[1]`.
34. When an array `msg` is created, its two values are set to `null` by default. So `msg[0]` has not been initialized (`null`) and you cannot call its methods.
35. In the best case, all the values in the shorter array are smaller than the first value in the longer array. We need 10 comparisons to establish that. In the worst case, neither array "runs out of values," so we need one comparison for each element of the resulting array, except the last one.
36. `mystery(0, 16)` prints 0, then calls `mystery(5, 15)`; that call prints 5, then calls `mystery(10, 14)`; that call prints 10, then calls `mystery(15, 13)`; that call prints 15 and quits.
37. There's nothing wrong with Choice A. Choice B works because `GOLDEN_RATIO` is a public static constant in `Pentagon`. In Choice C, `(1 + Math.sqrt(5.0))` is a `double`, so the arithmetic works.
38. One of the versions of Selection Sort finds the largest element among n elements, then among the remaining $n-1$ elements, and so on.
39. `n` remains greater than or equal to 2, until it is eventually reduced to either 0 or 1.

40. A call to Counter's `setCount` method has no effect, because `setCount` introduces and sets a local variable `count`, as opposed to modifying the instance variable `count`. The code prints 1. The correct method would be

```
public void setCount(int c) { count = c; } // Line 3
```

41. `isGood` returns `true` if the values in the first column are increasing and the values in each row are increasing.
42. It makes sense to sort the table first by distance, in ascending order, because you will get only a small number of rows at the top with distance ≤ 10 . Sorting by the number of reviews doesn't help much because most of them are big numbers. One or two rows may show up with fewer than 12 reviews, but they can easily be ignored. Sorting by rating, in descending order, is more helpful.

Answers and Solutions

Exam #3 ~ Free Response

1. (a)

```
public String chooseWinner()
{
    String record = "";

    while (record != null)
    {
        record = nextRecord();

        if (record != null)
        {
            recordNumber++;
            double prob = 1.0/recordNumber;

            if (Math.random() <= prob)
                winner = record;
        }
    }

    return winner;
}
```

(b)

```
public String reportWinner()
{
    int year = currentYear();

    int winnerBirthYear = Integer.parseInt(
        winner.substring(winner.length() - 4));
    int age = year - winnerBirthYear;

    winner = winner.substring(0, winner.length() - 11);

    int i = winner.indexOf(".");
    if (i != -1)
    {
        winner = winner.substring(0, i-2) +
            winner.substring(i+1);
    }

    return "And the winner is... " + winner + ", age " + age;
}
```

2.

```
public class CarMileage
{
    private final double TANK_SIZE;
    private int totalMilesDriven;
    private double totalGasUsed;

    public CarMileage(double tankSize)
    {
        TANK_SIZE = tankSize;
        totalMilesDriven = 0; // optional (default)
        totalGasUsed = 0;    // optional (default)
    }

    public int milesPerGallon(int miles, double tankFraction)
    {
        totalMilesDriven += miles;
        double gas = tankFraction * TANK_SIZE;
        totalGasUsed += gas;

        return (int)(totalMilesDriven / totalGasUsed + 0.5);
    }
}
```


3.

```
public int suspendOrders()
{
    int minIndex = 1;
    int minSum = sales.get(1) + sales.get(2);

    for (int k = 2; k < sales.size() - 2; k++)
    {
        int sum = sales.get(k) + sales.get(k+1);
        if (sum <= minSum)
        {
            minSum = sum;
            minIndex = k;
        }
    }

    int portion1 = minSum/2;
    int portion2 = minSum - portion1; 1
    sales.set(minIndex - 1, sales.get(minIndex - 1) + portion1); 2
    sales.set(minIndex + 2, sales.get(minIndex + 2) + portion2);
    sales.set(minIndex, 0);
    sales.set(minIndex + 1, 0);

    return minIndex;
}
```

Notes:

1. This way, `portion1 <= portion2`, as requested in the question.
2. Not `sales.get(minIndex - 1) + portion1` without `set`, of course.

4.

```
public boolean isComplete(int[][] m)
{
    int n = m.length;

    for (int i = 0; i < n; i++)
    {
        if (m[i][i] != 0)
            return false;
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (m[i][j] != 1 || m[j][i] != 1)
                return false;
        }
    }

    return true;
}
```

Answers and Solutions

Exam #4 ~ Multiple Choice

1. B	11. D	21. B	31. D	41. C
2. C	12. A	22. A	32. C	42. B
3. A	13. B	23. A	33. D	
4. C	14. C	24. B	34. C	
5. C	15. A	25. A	35. C	
6. A	16. C	26. D	36. C	
7. B	17. A	27. D	37. A	
8. A	18. A	28. B	38. B	
9. B	19. D	29. B	39. B	
10. A	20. D	30. C	40. D	

Notes:

1. `pos = url.indexOf(...)` sets `pos` to 0; `url.substring(0, 0)` returns an empty string.
2. Either way, $a > 10$. Try $a = 15$ with $b = 10$ and $b = 20$.
3. `% 100` leaves the last 2 digits.
4. We can safely distribute 2 over the “+” sign in parentheses, resulting in
$$2*(1) + 2*((3*b)/a) = 2 + 2*((b*3)/a), \text{ Choice C.}$$
5. Use a De Morgan’s Law: $!(x > y \ \&\& \ x \% y \neq 0)$ is the same as
 $(x \leq y \ || \ x \% y == 0)$.

6. The code was meant to be

```
if (fav >= 0)
{
    if (b > fav)
        System.out.println("IT IS MY FAVORITE BOOK, TOO");
    else if (m > fav)
        System.out.println("IT IS MY FAVORITE MOVIE, TOO");
}
else if (str.indexOf("NAME") > 0)
    System.out.println("MY NAME IS CHITCHAT");
```

Without this additional pair of braces added above, the code is treated as

```
if (fav >= 0)
{
    if (b > fav)
        System.out.println("IT IS MY FAVORITE BOOK, TOO");
    else if (m > fav)
        System.out.println("IT IS MY FAVORITE MOVIE, TOO");
    else if (str.indexOf("NAME") > 0)
        System.out.println("MY NAME IS CHITCHAT");
}
```

This error is known as the “dangling else” error. Since `fav = -1`, nothing is printed.

7. `\` and `\n` are one character each, so “S” is the fifth character, and its index is 4.
8. When an `int` value gets out of range, it might be interpreted as negative when the sign bit gets set to 1. In Java the range of integers is the same on all platforms — four bytes — so Choice D is wrong.
9. In `Math.pow(2, 3)`, 2 and 3 are promoted into doubles; the result is a double equal to $2^3 = 8$.
10. It helps to negate each of the expressions (using a De Morgan’s Law when necessary) and check whether the result is equivalent to “all three values are the same.” In Choice A, `!(a != b || b != c)` is the same as `(a == b && b == c)`, which is true if and only if `a == b` and `b == c` and `a == c`.
11. $5*4/2 = (5*4)/2$ evaluates to 10. $5/2*4 = (5/2)*4$ evaluates to 8. The result is converted into a double at the end.
12. `str1` and `str2` hold the same values if and only if `str1.equals(str2)` is true (`equals` is a boolean method) or `str1.compareTo(str2) == 0`. The expression in Choice C compares the references (basically, the addresses) of the strings, not their values. This might work sometimes for literal strings, but not always, so it is an error.
13. The sequence of values of `k` in the `for` loop is 1, 2, 4, 8.
14. Just google “API.” (The first public release of the CED in March 2025 has typos on pages 17, 26, and 39, incorrectly calling it “Application *Program* Interface”; on page 39, the error is only in the topic title, as the “Essential Knowledge” paragraph correctly uses “application programming interface.”)

15. Strings are immutable, so the call `process(word)` has no effect: `word` remains unchanged.
16. `mat[0][0]` remains equal to 2; `mat[1][1]` becomes $2*2 + 1 = 5$; `mat[2][2]` becomes $2*5 + 2 = 12$.
17. The code in Choice A won't work: `swap` receives and swaps copies of references to the objects. An array is passed to a method as a copy of a reference to the original array, so both Choices B and C will work.
18. $17/5$ gives 3; $3 \% 3 = 0$; $85 \% 3 = 1$.
19. `countSomething` returns the number of times the maximum value occurs in the array. If a new maximum is found, `count` is reset to 1.
20. In the situation in Choice A, the program throws a `FileNotFoundException`. In the situation in Choice B, a `NoSuchElementException` occurs. In the situation in Choice C, an `InputMismatchException` occurs.
21. The code first adds "0", "1", "2", ..., "9" to `digits`, then replaces each of the 5 pairs of elements "a", "b" with "a+b".
22. Before the loop, `words` contains ["ban", "an", "a"]; the `for` loop concatenates the elements to make "banana". The first occurrence of "an" in "banana" is at index 1.
23. The indices are used incorrectly in Choices C and D, and the indices are 199, 0 in Choice B.
24. `Test`'s constructor has a bug: it declares and sets a local variable `t`, not the class's instance variable `t` as intended. The class's instance variable remains equal to 0.
25. Every third element from 3 to 48 is set to 0 — 16 elements. Every fifth element from 5 to 50 is set to 0 — 10 elements. The total is $10 + 16 = 26$. But we counted the 15th, the 30th, and the 45th elements twice. $26 - 3 = 23$.
26. The last element of `list` is set in turn to "A", then to "B", then to itself — "B" again.
27. The `File`'s constructor only creates an internal description of the file; it doesn't open the file and never throws an exception (unless the argument is `null`). It is the `Scanner`'s constructor that throws an `IOException` (more precisely, an object of its subclass, `FileNotFoundException`) if the file is not found. (The first release of the 2025 CED — in the "Essential Knowledge" items 4.6.A.3 and 4.6.A.4 on page 105 — got it backwards.)
28. To compare two strings you must use `compareTo`, which returns an `int`, not necessarily 1.
29. $40 \text{ ms} + 40 \text{ ms}$ to sort both halves, plus $0.01 * 2000 = 20 \text{ ms}$ to merge them.

30. Let's trace the sequence of operations:

i	fibs[i]	n	s
7	34	30	"0"
6	21	9	"01"
5	13	9	"010"
4	8	1	"0101"
3	5	1	"01010"
2	3	1	"010100"
1	2	1	"0101000"
0	1	1	"01010001"

(Any positive integer can be represented uniquely in this "Fibonacci" number system as a sequence of binary digits with no two consecutive 1's in a row.)

31. `ArrayList` is implemented using a regular one-dimensional array. Once that array is full (that is, once the capacity of an `ArrayList` is reached), a bigger underlying array twice the size of the old array is created, the values from the old array are copied into the new one, and the old array is discarded.
32. `fun("ABCDE")` recursively calls `fun("CDE")`, giving `"E" + "CD"`, then appends `"AB"` at the end.
33. The three `FrequentFlyer` objects are created with the sum of their miles equal to 30000. The for-each loop gives 1000 additional miles to each of them. It is often said that you cannot modify the elements of a list (an array or an `ArrayList`) within a for-each loop. It is true that you cannot replace an element in the list, because the reference to that element cannot change in a for-each loop. But if the list holds a mutable object, you can change the state of that object by calling one of its mutator methods.
34. On each iteration through the `while` loop, the code takes the elements from index 0 through index `n-1` and repeats that segment of the array, adding 1 to each repeated element. Thus `{0}` becomes `{0, 1}`, then `{0, 1}` becomes `{0, 1, 1, 2}`, then `{0, 1, 1, 2}` becomes `{0, 1, 1, 2, 1, 2, 2, 3}`.
- (Notice that `nums[k]` is the number of binary digits equal to 1 in the binary representation of `k`. The sequence can grow in the same manner up to any power of 2. The infinite sequence formed like this is a "fractal" sequence: if you take every other element starting from 0, you will get the same sequence!)
35. `mystery(6)` returns `6 + mystery(4)`; `mystery(4)` returns `4 + mystery(2)`; `mystery(2)` returns `2 + mystery(0)`; `mystery(0)` returns 0. $6 + 4 + 2 + 0 = 12$.
36. `a` and `b` are references to the same array; `c` is a reference to a different array with the same values as the initial values in `a`. When `a` changes, `c` remains unchanged.
37. `[] => [1] => [1, 2] => [1, 1, 2] => [1, 1, 2, 2] => [1, 1, 2, 3, 2]`.
38. In Choice B, after the comparison with the value 7 at the center, the search will continue in the left half of the array.
39. The elements in the 2-by-2 square in the middle of the larger square are set to 1.

- 40. The concept of privacy applies to the class as a whole. Any method of a class has access to private instance variables of all objects of that class.
- 41. Each of the 3 elements equals only itself.
- 42. As explained in the question, the `Scanner` object `input` behaves as though it were created with a file containing only one token: `Names.txt`. This string is returned by the `nextLine` method, and the `split` method returns an array with only one element. The `for` loop is never entered.

Answers and Solutions

Exam #4 ~ Free Response

1. (a)

```
public String suggestStrongPassword()
{
    String password = "";

    password += randomChar(ABC); 1

    for (int k = 1; k <= 3; k++)
    {
        password += randomChar(digits);
    }

    for (int k = 1; k <= 7; k++)
    {
        password += randomChar(abc);
    }

    password += randomChar(symbols);

    return password;
}
```

Notes:

1. Call the provided method `randomChar`.

(b)

```
public String makePassword(String phrase, int n)
{
    String password = phrase.substring(0, 1);

    for (int i = 1; i < phrase.length(); i++)
    {
        if (phrase.substring(i, i+1).equals(" ")) 1
            password += phrase.substring(i+1, i+2);
    }

    while (n != 0)
    {
        password += n % 10;
        n /= 10;
    }

    return password;
} 2
```

Notes:

1. In `str.substring(i, j)`, it is okay to have `j` equal to `str.length()` — it does not cause a `StringIndexOutOfBoundsException`.
2. Alternative solutions:

```
...
String password = phrase.substring(0, 1);

int i = 1;
while (i != -1)
{
    i = phrase.indexOf(" ", i);
    if (i != -1)
    {
        i++;
        password += phrase.substring(i, i+1);
    }
}
...
```

Or

```
...
String password = "";
String[] words = phrase.split(" ");

for (String word : words)
    password += word.substring(0, 1);
...
```

2.

```
public class ApplesPurchase
{
    private int applePrice;
    private int qtyPurchased;

    public ApplesPurchase(int cents, int howMany)
    {
        applePrice = cents;
        qtyPurchased = howMany;
    }

    public String totalCost()
    {
        int sixes = applesQty / 6;
        int cents = applePrice * (applesQty - sixes); 1

        int dollars = cents/100;
        cents -= 100*dollars;

        String dollarsAndCents = "$" + dollars + ".";
        if (cents < 10)
            dollarsAndCents += "0";
        dollarsAndCents += cents;

        return applesQty + " apples at " + applePrice +
            "c each: " + dollarsAndCents + " total";
    }
}
```

Notes:

1. This is an easy way to adjust the price for the six-for-five deal.

3.

```
public double averageScore(ArrayList<Integer> scores)
{
    int sum = 0, count = 0;
    int maxScore = 0;

    for (Integer sc : scores)
    {
        sum += sc;
        count++;
        if (sc > maxScore)
            maxScore = sc;
    }

    if (scores.size() >= 6 &&
        countOccurrences(scores, maxScore) == 1) 1
    {
        sum -= maxScore;
        count--;
    }
    return (double)sum / count;
}
```

Notes:

1. Call the provided method `countOccurrences`.

4.

```
public Location findBestPairOfSeats()
{
    Location optimal = new Location(3, seats[0].length/2 - 1);
    // the left seat of the optimal pair
    int minDistance = seats.length + seats[0].length;
    // or any big number
    Location bestLocation = null;

    for (int r = 2; r < seats.length; r++) 1
    {
        for (int c = 0; c < seats[0].length - 1; c++)
        {
            if (seats[r][c] == false && seats[r][c+1] == false)
            {
                Location loc = new Location(r, c);
                int distance = loc.getDistance(optimal);

                if (distance < minDistance)
                {
                    minDistance = distance;
                    bestLocation = loc;
                }
            }
        }
    }
    return bestLocation;
}
```

Notes:

1. Recall that in an array, `length` is not a method.

Index

- ! logical operator, 19
- % operator, 16
- & operator, 22
- && operator, 19
- ^ operator, 22
- | operator, 22
- || operator, 19
- ~ operator, 22
- ++ and -- operators, 17
- +=, -=, */, /=, and %= operators, 3, 17
- == and != operators, 18
 - for objects, 18
- abs methods of Math, 40, 63, 64, 71
- accessor methods, 5, 54, 58, 59
- add methods of ArrayList, 45
- algorithm, 81
- AP credit, 8
- AP CSA Development Committee, vii, ix
- AP Java subset, viii, 3
- API documentation, 76
- Application Programming Interface (API), 76
- argument vs. parameter, 56, 57
- arithmetic operators, 15
- ArithmeticException, 4, 23, 47
- array algorithms, 6
- ArrayIndexOutOfBoundsException, 4, 38, 47
- ArrayList algorithms, 6
- ArrayList class, 4, 44
- ArrayList methods, 44
- arrays, 37
 - one-dimensional, 37
 - index, 37
 - length, 37
 - passed to a method, 67
 - returning from methods, 41
 - traversal, 81
 - two-dimensional, 41
- artificial intelligence (AI), 77
- assignment operators, 3
- autoboxing, 5, 36, 37, 44
- base case in recursion, 95
- beprepared website, ix
- Binary Search, 6, 88
 - recursive, 97
- bitwise logical operators, 22
- Bluebook, viii, 1, 10
- boolean data type, 13
- Boolean expressions, 68
- boundary conditions, 87
- break in loops, 4, 5, 29
- cast operator, 15
- CED (Course and Exam Description), vii
- char data type, 3
- checked exceptions, 5, 47, 52

- class, 55
 - accessor methods, 58
 - client of, 58, 73
 - constructors, 56
 - instance variables, 58
 - mutator methods, 59
 - public and private members, 58
 - static methods, 61, 64
 - static (class) variables, 61, 62
- class free-response question, 78
- class hierarchy, viii, 79
- client of a class, 58, 73
- code efficiency, 9
- comments, 11, 55
- compareTo method of String, 18, 33
- compile-time errors, 38
- compound assignment operators, 3, 17
- concatenating strings, 3, 31
- ConcurrentModificationException, 47
- conditional statements, 22
- constructors, 55, 56
- continue statement in loops, 5, 30
- Course and Exam Description (CED), vii
- cut-off points, 8
- data abstraction, 77
- data collection, 77
- data structure, 77
- data types in arithmetic operators, 15
- De Morgan's Laws, 20
- Development Committee, vii, ix
- divide and conquer algorithms, 88, 98
- Double class, 35
- double data type, 13
- doubleValue method of Double, 4, 35
- do-while loop, 5
- enum data types, 5
- equals method, 18
 - of the Object class, 79, 80
- equals vs. == for objects, 18
- escape character \n, 3
- escape characters in literal strings, 3, 31
- exam
 - class free-response question, 78
 - format, 1
 - free-response questions, viii, 1
 - grade distributions, 6
 - grading, 6
 - multiple-choice questions, 2
 - rubric, 7
- exam-taking tips, 9
- exceptions, 4, 38, 47
- File class, 3, 5, 51, 52, 76
- final keyword, 15
- floating-point numbers, 15
- for loop, 4, 24
- for-each loop, 4, 27, 46, 82
 - with 2D arrays, 43
- formatting output, 49
- free-response questions, viii, 1
- fully-qualified name, 76
- functional decomposition, 77
- garbage collection, 37, 57
- get method of ArrayList, 45
- grade distributions, 6
- grading, 6
- has-a relationship between classes, 79
- hierarchy of classes, viii, 79
- if-else statement, 3
- if-else-if sequences, 22
- IllegalArgumentException, 61
- immutable objects, 67
- import statements, 11, 76

- increment and decrement operators, 3, 17
- indexOf method of String, 33
- IndexOutOfBoundsException, 4, 45, 47
- indices, 37
- information hiding, 58, 77
- inheritance, viii, 79
- InputMismatchException, 4, 52
- inserting a value into an array, 86
- Insertion Sort, 6, 86, 92
- instance of a class, 55
- instance variables, 5, 55, 58
- instantiation, 55
- int data type, 13
- Integer class, 35
- Integer.MAX_VALUE, 3, 4, 13, 16, 36
- Integer.MIN_VALUE, 3, 4, 13, 16, 36
- intValue method of Integer, 4, 35, 36
- IOException, 3, 4, 5, 47, 51, 52
- is-a relationship between classes, 79
- iterations, 81
-
- Java Development Kit (JDK), 76
- Java Quick Reference, 76
- Java subset, viii, 3
- java.lang package, 76
- javadoc comments, 56
- JDK (Java Development Kit), 76
-
- Learning Objectives, vii
- length of an array, 37
- library classes in the subset, 4
- linear time, 92
- LinkedList class, 44
- literal strings, 31
- local variables, 5, 13, 74
- logical operators, 3, 19
- loops, 81
-
- Math methods, 63, 64
 - Math.abs, 40, 63, 64, 71
 - Math.max, 4, 65
 - Math.min, 4, 65
 - Math.pow, 63
 - Math.random, 63, 75
 - Math.sqrt, 15, 63
- max method of Math, 4
- Math.PI and Math.E constants, 64
- MAX_VALUE in Integer class, 3, 4, 13, 16, 36
- maximum or minimum
 - finding in an array, 82
- Mergesort, 6, 98
- method signature, 5
- methods, viii, 1, 2
 - accessor, 5, 54, 58, 59
 - mutator (modifier), 5, 59, 67, 74
 - naming convention, 64
 - overloaded, 70
 - static, 61, 64
- min method of Math, 4, 65
- MIN_VALUE in Integer class, 3, 4, 13, 16, 36
- modifier methods, 5, 74
- modulo division operator %, 16
- multiple-choice questions, 1, 2
- mutator methods, 5, 59, 67
-
- nested ifs, 22
- nested loops, 29, 81
- new operator, 4, 57
- no-args constructor, 44, 57
- null, 18, 70
- NullPointerException, 4, 18, 47, 48
- Object class, 79
- object-oriented programming, viii, 79
- objects, 4, 18
 - immutable, 67
 - passed to methods, 67

off-by-one bugs (OBOBs), 26
one-dimensional arrays, 3, 37
OOP (Object-Oriented Programming),
 viii, 79
output formatting, 49
overloaded methods, 70

parameters, 5
passed by value, 66
parentheses, use of, 21
parseInt method of Integer, 35
passing arguments by value, 5
pathname, 51
PI constant of Math, 64
polymorphism, viii
postcondition, 3, 56
pow method of Math, 63
precedence of operators, 21
precondition, 3, 56
primitive data types, 3, 13, 66
printf method, 50
private, 58
procedural abstraction, 77
programming style, 78
public and private class members, 58

quadratic sorts, 91
quadratic time, 92
Quick Reference, 1

random method of Math, 63, 75
random numbers, 75
reading text files, 51
recursion, 93
 base case, 95
recursive Binary Search, 97
recursive code, 6
regular expressions, 33
relational operators, 3, 18
remove method of ArrayList, 45

return statement, 65, 68
 in loops, 29
rounding, 5
rubric, 7
run-time errors, 38, 47

Scanner class, 3, 5, 51
 close method, 52
 constructor, 51
 hasNext method, 52
 next method, 52
 nextBoolean method, 52
 nextDouble method, 52
 nextInt method, 52
 nextLine method, 52
Selection Sort, 6, 91
Sequential Search, 6, 88
set method of ArrayList, 45
short-circuit evaluation, 20, 22
size method of ArrayList, 45
skylit.com/beprepared website, ix
sorting, 91
specifications, 77
split method of String, 33
sqrt method of Math, 15, 63
static final variables, 15
static keyword, 61
static methods, 61, 64
static variables, 61, 62
String class, 31
 compareTo method, 3
 concatenation, 3, 31
 equals method, 3
 immutability, 31, 32
 indexOf method, 3
 length method, 3
 split method, 3
 substring method, 3
String methods, 32
StringIndexOutOfBoundsException,
 33, 47

- subclass, 79
- subset, vii, ix, 1
 - input and output, 5
 - library classes, 4
 - primitive data types, 3
- String methods, 3
- substring methods of String, 33, 71
- superclass, 79
- switch statement, 5
- symbolic constants, 14, 15
- System.out.print, 3, 11, 49, 50
- System.out.printf, 50
- System.out.println, 3, 11, 49, 50
- this keyword, 59, 63
- throwing an exception, 47
- toString method, 50, 80
 - of the Object class, 79, 80
- traversing an array, 81
- triangular nested loops, 82
- true and false keywords, 3
- try-catch statement, 5, 47
- two-dimensional arrays, 3, 41
- unboxing, 5, 36
- unchecked exceptions, 47
- void methods, 64
- while loop, 24
- whitespace characters, 54
- wrapper classes, 35

Index to Free-Response Questions in Practice Exams

	Exam	Question	Page	Solution
Apflix movie matching	2	3	163	244
Apples purchase	4	2	222	263
Ballroom competition scoring	4	3	224	264
Car gas mileage	3	2	193	252
Dorm rooms	1	1	127	233
EGOT status	2	1	159	241
Graphs	3	4	197	254
Highway	1	4	135	236
Memory Lane game	1	3	132	235
Passwords	4	1	219	261
Reviewed product	1	2	130	234
Sales history	3	3	194	253
Shopping options	2	2	162	243
Sliding Tiles puzzle	2	4	166	245
Sweepstakes	3	1	189	251
Theater seats	4	4	226	265

NOTES
